

AGENTS.md — Руководство для AI-агентов и разработчиков

Этот документ — единая точка входа для **AI-агентов** (Claude, Cursor, Copilot, Codex и др.) и **разработчиков**, подключающихся к проекту **Multichannel Hub** на любом этапе. Он описывает архитектуру, конвенции кода, рабочие процессы и пошаговые рецепты доработки.

✚ Прочитайте этот файл **полностью** перед внесением изменений. Он дополняет [README.md](#) (./README.md) (запуск), [API.md](#) (./API.md) (REST API) и [INTEGRATION.md](#) (./INTEGRATION.md) (интеграции и добавление каналов).

1. Что это за проект

Multichannel Hub — омниканальный шлюз (агрегатор каналов связи). Принимает обращения из разных источников (Telegram, веб-формы и др.), сохраняет их в единую ленту и пересылает во внешние системы (CRM, n8n, Make, Bitrix24, Zapier) через вебхуки.

Ключевой принцип архитектуры — модульность. Новый канал связи добавляется как отдельный NestJS-модуль без изменения ядра системы. Не нарушайте этот принцип.

Технологический стек

Слой	Технологии
Backend	NestJS 10, TypeScript, Prisma ORM, PostgreSQL, BullMQ + Redis, Swagger
Frontend	Next.js 14 (App Router), React 18, Tailwind CSS, кастомный i18n
Инфра	Docker, Docker Compose

2. Карта репозитория

```

multichannel-hub/
├── backend/                                # NestJS API (порт 4000, префикс /api)
│   ├── src/
│   │   ├── auth/                         # JWT-авторизация (login, guard, strategy)
│   │   ├── users/                       # Профиль, смена пароля
│   │   ├── channels/                   # CRUD каналов (общий для всех типов)
│   │   ├── telegram/                 # Канал: Telegram Bot API
│   │   ├── webforms/                 # Канал: приём заявок с веб-форм
│   │   ├── webhooks/                 # Доставка событий вовне (+ BullMQ processor)
│   │   ├── messages/                 # История сообщений и статистика
│   │   ├── settings/                 # Глобальные настройки (key-value)
│   │   ├── prisma/                   # PrismaService (доступ к БД)
│   │   ├── app.module.ts              # Корневой модуль – РЕГИСТРАЦИЯ всех модулей
│   │   └── main.ts                   # Bootstrap: CORS, ValidationPipe, Swagger
│   └── prisma/
│       ├── schema.prisma              # Модель данных (источник истины для БД)
│       ├── migrations/               # SQL-миграции
│       └── seed.ts                   # Создание админа по умолчанию
├── frontend/                             # Next.js админ-панель (порт 3000)
│   └── src/
│       ├── app/
│       │   ├── login/                 # Страница входа
│       │   └── (panel)/               # Защищённые страницы (dashboard, channels, ...)
│       ├── components/               # Sidebar, LanguageSwitcher
│       └── lib/                      # api.ts (HTTP-клиент), i18n.ts, LangContext.tsx
├── docker-compose.yml                 # Оркестрация: postgres, redis, backend, frontend
└── README.md / API.md / INTEGRATION.md / AGENTS.md

```

Где что искать

- **Добавить/изменить таблицу БД** → `backend/prisma/schema.prisma` (+ миграция).
- **Зарегистрировать новый модуль** → `backend/src/app.module.ts`.
- **Глобальные настройки приложения** (CORS, Swagger, валидация) → `backend/src/main.ts`.
- **Тексты интерфейса / переводы** → `frontend/src/lib/i18n.ts`.
- **HTTP-клиент фронтенда** (подстановка JWT-токена) → `frontend/src/lib/api.ts`.
- **Навигация в панели** → `frontend/src/components/Sidebar.tsx`.

3. Запуск окружения

Через Docker (рекомендуется для проверки целиком)

```

cp .env.example .env
docker compose up -d --build

```

Сервис	URL
Админ-панель	http://localhost:3000
API	http://localhost:4000/api
Swagger	http://localhost:4000/api/docs

Учётные данные по умолчанию: `admin@admin.com` / `admin123`.

Локальная разработка (быстрый цикл)

```
# Backend
cd backend && cp .env.example .env && npm install
npx prisma migrate dev && npm run prisma:seed
npm run start:dev          # hot-reload на :4000

# Frontend (отдельный терминал)
cd frontend && cp .env.example .env && npm install
npm run dev                # hot-reload на :3000
```

⚠ Для локального запуска без Docker в `backend/.env` замените хосты `postgres` / `redis` на `localhost`, а в `DATABASE_URL` укажите доступную базу PostgreSQL.

4. Конвенции кода (обязательны)

Общие

- **Язык комментариев, сообщений об ошибках и UI-текстов — русский.** Имена переменных, функций, классов и файлов — английский.
- **TypeScript строгий.** Не используйте `any` без необходимости; типизируйте DTO и ответы.
- Не коммитьте секреты. Все настройки — через переменные окружения и `.env` (см. `.env.example`).
- `.env` в `.gitignore`; шаблон храните в `.env.example`.

Backend (NestJS)

- Структура модуля стандартна: `<feature>.module.ts`, `<feature>.controller.ts`, `<feature>.service.ts`, плюс `dto/` для входных данных.
- **Каждый новый модуль регистрируется в `app.module.ts`** в массиве `imports`.
- Входные данные валидируются через DTO + `class-validator`. Глобальный `ValidationPipe` с `whitelist: true` уже включён в `main.ts` — поля без декораторов отсекаются.
- Доступ к БД — только через `PrismaService` (`backend/src/prisma`). Не создавайте отдельные подключения к Prisma.
- Защищённые эндпоинты помечаются `@UseGuards(JwtAuthGuard)`. Публичные (входящие вебхуки Telegram, приём веб-форм) — намеренно без guard, доступ контролируется токеном в URL.
- Все REST-методы документируются Swagger-декораторами (`@ApiTags`, `@ApiOperation` и т.д.).

- Длительные/ненадёжные операции (доставка вебхуков) выполняются через **очередь Bull-MQ**, а не синхронно в запросе. См. `webhooks.service.ts` → `dispatch()` и `webhook.processor.ts`.

Frontend (Next.js App Router)

- Страницы панели лежат в группе `src/app/(panel)/` и автоматически защищены layout-компонентом (`layout.tsx` проверяет JWT в `localStorage`).
- Все запросы к API — через клиент `src/lib/api.ts` (он сам подставляет `Authorization`). Не используйте «голый» `fetch` / `axios` напрямую в компонентах.
- **Любой видимый текст добавляется в словари `src/lib/i18n.ts` для всех трёх языков (`ru`, `en`, `es`) и выводится через хук `useLang()`.** Хардкод строк запрещён.
- Стилизация — только Tailwind CSS (утилитарные классы), без отдельных CSS-файлов на компонент.

5. Модель данных (Prisma)

Источник истины — `backend/prisma/schema.prisma`. Основные таблицы:

Модель	Назначение
User	Пользователи панели. Роли <code>ADMIN</code> / <code>OPERATOR</code> , язык интерфейса.
Channel	Канал связи. Поле <code>type</code> (<code>ChannelType</code>) + <code>config: Json</code> (токены и пр.).
Message	Входящие/исходящие сообщения. <code>direction</code> , <code>status</code> , <code>payload: Json</code> .
Webhook	Исходящие вебхуки: <code>url</code> , <code>secret</code> , массив <code>events</code> .
Setting	Глобальные настройки в формате ключ-значение.

Расширяемость каналов заложена в enum `ChannelType`:

`TELEGRAM`, `WEBFORM`, `WHATSAPP`, `VK`, `MAX`, `INSTAGRAM`. Значения уже присутствуют — при добавлении нового канала бизнес-логику пишите в новом модуле, а специфичные данные канала храните в `Channel.config` (JSON), не плодя новые колонки без необходимости.

Изменение схемы — порядок действий

```
cd backend
# 1. Отредактируйте schema.prisma
# 2. Создайте миграцию (имя — по сути изменения)
npx prisma migrate dev --name add_xxx
# 3. Регенерируйте клиент (обычно делается автоматически)
npm run prisma:generate
```

Миграции **коммитятся** в репозиторий. В Docker/проде применяются через `npm run prisma:migrate ( prisma migrate deploy )`.

6. Рецепт: добавить новый канал связи

Это самая частая задача. Пример — канал `WHATSAPP` (**уже реализован**, см. `backend/src/whatsapp/`).

Для добавления нового канала (например, VK или MAX):

1. **Schema**: убедитесь, что нужное значение есть в `enum ChannelType`. Если нет — добавьте и создайте миграцию (см. раздел 5).
2. **Модуль backend**: создайте папку `backend/src/whatsapp/` по образцу `telegram/`:
 - `whatsapp.module.ts` — импортирует `ChannelsModule` и `MessagesModule` (см. как сделано в `telegram.module.ts`).
 - `whatsapp.service.ts` — логика приёма/отправки. Входящее сообщение сохраняйте через `MessagesService`, затем вызывайте `webhooksService.dispatch('message.received', payload)` для пересылки во внешние системы.
 - `whatsapp.controller.ts` — публичный эндпоинт приёма входящих (вебхук провайдера) и защищённые эндпоинты управления.
 - `dto/` — DTO с валидацией для всех входных данных.
3. **Регистрация**: добавьте `WhatsappModule` в `imports` в `backend/src/app.module.ts`.
4. **Frontend**: на странице `channels/page.tsx` добавьте поддержку нового типа в форме создания канала; новые тексты — в `i18n.ts` (ru/en/es).
5. **Документация**: опишите канал в `README.md` и `INTEGRATION.md`.
6. **Проверка**: соберите backend (`npm run build`) и прогоните сценарий через Swagger.

💡 Предпочитайте **бесплатные / неофициальные** способы интеграции (по аналогии с подходом Evolution API для WhatsApp), если официальный API платный — это требование продукта. Изолируйте такой код в сервисе канала, чтобы его можно было заменить.

7. Рецепт: добавить REST-эндпоинт

1. Опишите/обновите DTO в `dto/` с декораторами `class-validator`.
2. Добавьте метод в сервис (вся бизнес-логика — в сервисе, не в контроллере).
3. Добавьте метод в контроллер с роутингом и Swagger-декораторами.
4. Для защищённого доступа — `@UseGuards(JwtAuthGuard)`.
5. Обновите `API.md` (`./API.md`).

8. Вебхуки и события

- Отправку наружу выполняет `WebhooksService.dispatch(event, payload)` — он ставит задачи в очередь BullMQ `webhooks`, а `webhook.processor.ts` доставляет их с ретраями (5 попыток, экспоненциальная задержка).
- Поддерживается подпись `X-Webhook-Signature` (HMAC-SHA256) при заданном `secret`.

- Текущее основное событие — `message.received`. Новые события называйте по схеме `<сущность>.<действие>` (например, `message.sent`, `channel.created`) и документируйте в `INTEGRATION.md`.

9. Сборка, проверка и Git-процесс

Перед коммитом обязательно

```
# Backend
cd backend && npm run build      # должен собираться без ошибок TypeScript
npm run lint                    # ESLint --fix

# Frontend
cd frontend && npm run build      # next build без ошибок
npm run lint
```

⚠ Автоматических тестов в проекте пока нет. До их появления **ручная проверка через Swagger (`/api/docs`) и UI обязательна**. Если добавляете тесты — используйте Jest (идёт с NestJS) и кладите рядом с кодом (`*.spec.ts`).

Ветки и Pull Request

- **Никогда не коммитьте напрямую в `main`**. Работайте в feature-ветке: `feature/<краткое-описание>`, `fix/<...>`, `docs/<...>`.
- Один PR — одна логическая задача. Описывайте, что и зачем изменено.
- Сообщения коммитов — в стиле Conventional Commits: `feat:`, `fix:`, `docs:`, `refactor:`, `chore:`. Кратко и по сути.
- **PR не мерджит сам агент** — слияние подтверждает владелец репозитория.

10. Чек-лист агента перед сдачей задачи

- [] Изменения в feature-ветке, не в `main`.
- [] `npm run build` проходит и в `backend`, и в `frontend`.
- [] Схема БД изменена вместе с миграцией (если затрагивалась).
- [] Новый модуль зарегистрирован в `app.module.ts`.
- [] Все новые UI-тексты добавлены в `i18n.ts` для `ru`, `en`, `es`.
- [] Новые/изменённые эндпоинты задокументированы в `API.md` и в Swagger.
- [] Секреты не попали в репозиторий; обновлён `.env.example` при новых переменных.
- [] Обновлена документация (`README.md` / `INTEGRATION.md`) при изменении поведения.
- [] Создан понятный PR; финальное слияние оставлено владельцу.

11. Чего делать НЕЛЬЗЯ

- ❌ Ломать модульность: складывать логику конкретного канала в ядро или в чужие модули.
- ❌ Обращаться к БД в обход `PrismaService`.

- ❌ Хардкодить тексты в компонентах вместо `i18n`.
- ❌ Делать тяжёлые/сетевые операции синхронно вместо очереди, где это уместно.
- ❌ Коммитить `.env`, токены, ключи.
- ❌ Мерджить в `main` без ревью владельца.
- ❌ Менять порты `1000` / `2200` и прочие зарезервированные платформой значения.

Поддерживайте этот файл в актуальном состоянии: при изменении архитектуры, конвенций или процессов обновляйте соответствующие разделы в том же PR.